

PERFORMANCE

Performance

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

This document is the reference for performance testing, load testing, stress testing, and the target service-level objectives (SLOs) for the merge service.

Test suites

pytest-benchmark (latency, microbench)

Directory: tests/benchmark/

File	Under test	Micro-metric
test_search_benchmark.py	SearchOrchestrator.search end-to-end	wall clock per call
test_deduplication_benchmark.py	Deduplicator.merge_results on synthetic result sets	wall clock + per-op rate

Invocation:

```
pytest tests/benchmark/ --benchmark-only -v
# Comparing against a saved baseline:
pytest tests/benchmark/ --benchmark-compare
```

Output lands in .benchmarks/ per pytest-benchmark's default layout and is uploaded by the nightly CI workflow.

Performance suite (concurrent behaviour)

Directory: tests/performance/

File	Scenario
test_concurrent_search.py	5–20 concurrent searches against the live stack; asserts no deadlocks, no result corruption, latency under the gate

Invocation:

```
pytest tests/performance/ -v --import-mode=importlib
```

Locust load profile

Directory: tests/load/ — **pending Phase 6.**

Planned user profiles:

Profile	Users	Spawn rate	Think time	Target endpoint
browser_user	50	1/s	2–5 s	POST /search, then SSE stream
api_user	50	2/s	0 s	POST /search, GET /search/{id}

Invocation once the suite exists:

```
locust -f tests/load/locustfile.py \  
  --host http://localhost:7187 \  
  --users 100 --spawn-rate 5 \  
  --run-time 10m --headless
```

Results land in artifacts/load/<timestamp>/.

Stress suite

Directory: tests/stress/

File	Scenario
test_search_stress.py	

File	Scenario
	Sustained 100–1000 VU load; asserts graceful degradation and recovery

Invocation:

```
pytest tests/stress/ -v --import-mode=importlib
```

Target SLOs

These come from the completion-initiative plan (Part D, Phase 6) and are the acceptance criteria for the service once Phase 6 ships.

SLO	Target	Measured by
p50 search latency at 10 VU	< 500 ms	tests/performance/ test_concurrent_search.py
p95 search latency at 100 VU	< 2000 ms	Locust (tests/load/)
Error rate at 100 VU	< 0.1 %	Locust
Graceful degradation at 1000 VU	no 5xx cascade; 429 or 503 returned	tests/stress/ + chaos probes
Recovery after chaos fault	< 60 s to resume p95 target	tests/stress/ + toxiproxy (tests/chaos/, pending)
SSE connection stability	> 95 % of streams complete	tests/integration/ test_realtime_streaming.py
Deduplicator throughput	> 10 000 results/sec	tests/benchmark/ test_deduplication_benchmark.py

Instrumentation

tests/instrumentation/ is reserved for cross-cutting perf instrumentation helpers (timers, scrape utilities). Empty today.

Metric sources

The p95 and recovery SLOs are measured from Prometheus:

```
# p95 search duration
histogram_quantile(
  0.95,
  sum by (le)
(rate(qbit_merge_search_duration_seconds_bucket[5m]))
)

# Active-searches recovery
qbit_merge_active_searches
```

See [OBSERVABILITY.md](#).

CI integration

- `nightly.yml` runs the benchmark suite and uploads the JSON to artifacts.
- A regression gate is **pending** — Phase 6 wires `pytest-benchmark's` `--benchmark-compare-fail=mean:10%` so a 10 % regression fails the build.

Gotchas

- The `pytest-timeout` global 30-second cap applies to benchmark tests unless they declare `@pytest.mark.timeout(300)`. Long-running stress tests will time out silently otherwise.
- Locust users reuse HTTP connections by default; set `LOCUST_ENABLE_CONN_POOLING=0` to force a fresh connection per request when reproducing session-handling bugs.
- When running against a live stack on the same host, CPU contention between the locust workers and the merge service skews p95. Prefer a two-host setup or constrain locust with `taskset`.
- Chaos probes (toxiproxy) require the `tests/chaos/` suite which is pending — stress runs today only simulate load, not network faults.